

Introdução e ambiente de programação

Rápida introdução à linguagem Python

- Criada por Guido van Rossum, no final dos anos 1980.
- Um objetivo era criar uma linguagem de uso genérico e de fácil acesso para iniciantes.
- Guido participou do trabalho de evolução da linguagem até 2018.
- Linguagem Open Source
- Comunidade forte e ativa.
- Simples em sua versão básica, porém apoiada por uma ampla disponibilidade de bibliotecas.

Deepnote

- O Deepnote é uma IDE online que traz funcionalidades a mais para o ambiente de desenvolvimento clássico do Jupyter Notebook/Google Colaboratory.
- Similar ao Jupyter Notebook offline ou Google Colab, porém, para saber mais sobre a ferramenta e seus recursos uma excelente fonte é a documentação oficial que está em [Deepnote Docs](#).

Referências para linguagem Jupyter Markdown

- [Jupyter Notebook Users Manual](#)
- [Markdown for Jupyter notebooks cheatsheet, IBM](#)

Zen do Python

The Zen of Python, by Tim Peters

```
Beautiful is better than ugly.  
Explicit is better than implicit.  
Simple is better than complex.  
Complex is better than complicated.  
Flat is better than nested.  
Sparse is better than dense.  
Readability counts.  
Special cases aren't special enough to break the rules.  
Although practicality beats purity.  
Errors should never pass silently.  
Unless explicitly silenced.  
In the face of ambiguity, refuse the temptation to guess.  
There should be one-- and preferably only one --obvious way to do it.  
Although that way may not be obvious at first unless you're Dutch.  
Now is better than never.  
Although never is often better than *right* now.  
If the implementation is hard to explain, it's a bad idea.  
If the implementation is easy to explain, it may be a good idea.  
Namespaces are one honking great idea -- let's do more of those!
```

Conceitos Básicos

Comentários e estilo

```
Hello World!
```

 Execution error

SyntaxError: invalid syntax (1870465760.py, line 1)

Show error details

```
'\nComentario de multiplas linhas\nlinha 1\nlinha 2\nlinha 3\n'
```

Variáveis e tipos

VARIÁVEIS: são abstrações de locais de armazenamento de dados. Os dados armazenados em uma VARIÁVEL podem mudar ao longo da execução de um programa. Daí o nome VARIÁVEL.

TIPO DE DADO: determina o significado dos valores achados em uma VARIÁVEL.

```
['False',  
 'None',  
 'True',  
 'and',  
 'as',  
 'assert',  
 'async',  
 'await',  
 'break',  
 'class',  
 'continue',  
 'def',  
 'del',  
 'elif',  
 'else',  
 'except',  
 'finally',  
 'for',  
 'from',  
 'global',  
 'if',  
 'import',  
 'in',  
 'is',  
 'lambda',  
 'nonlocal',  
 'not',  
 'or',  
 'pass',  
 'raise',  
 'return',  
 'try',
```

 Execution error

SyntaxError: invalid decimal literal (3494753308.py, line 7)

Show error details

```
1  
False
```

```
12  
25
```

```
30
```

Algumas Funções Úteis

O interpretador Python possui diversas funções e tipos integrados que estão sempre disponíveis.

```
1
1.3
True
Introducao a programacao com python
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'str'>
```

```
<class 'int'>
<class 'bool'>
```

Help on int object:

```
class int(object)
| int([x]) -> integer
| int(x, base=10) -> integer
|
| Convert a number or string to an integer, or return 0 if no arguments
| are given. If x is a number, return x.__int__(). For floating point
| numbers, this truncates towards zero.
|
| If x is not a number or if base is given, then x must be a string,
| bytes, or bytearray instance representing an integer literal in the
| given base. The literal can be preceded by '+' or '-' and be surrounded
| by whitespace. The base defaults to 10. Valid bases are 0 and 2-36.
| Base 0 means to interpret the base from the string as an integer literal.
| >>> int('0b100', base=0)
| 4
|
| Built-in subclasses:
|     bool
|
| Methods defined here:
|
| __abs__(self, /)
|     abs(self)
|
| __add__(self, value, /)
|     Return self+value.
|
| ...and (self, value, /)
```

```
[ '__add__',
  '__class__',
  '__contains__',
  '__delattr__',
  '__dir__',
  '__doc__',
  '__eq__',
  '__format__',
  '__ge__',
  '__getattr__',
  '__getitem__',
  '__getnewargs__',
  '__getstate__',
  '__gt__',
  '__hash__',
  '__init__',
  '__init_subclass__',
  '__iter__',
  '__le__',
  '__len__',
  '__lt__',
  '__mod__',
  '__mul__',
  '__ne__',
  '__new__',
  '__reduce__',
  '__reduce_ex__',
  '__repr__',
  '__rmod__',
  '__rmul__',
  '__setattr__',
  '__sizeof__',
```

PEP 8 – Guia de Estilo para Código Python

A convenção de código Python é regida pela [PEP 8](#), focando em legibilidade e consistência.